HINTS COMPANION

neue fische

Performing Reconnaissance — Hints Companion

Per-task hints, samples, and troubleshooting

Your role: Junior Cybersecurity Analyst
Companion to: *Performing Reconnaissance — Learner Lab Document* (file 08)

How to Use This Document

This document is your guided companion to the *Performing Reconnaissance* lab. It is keyed to the same 12 tasks as the Lab Document, in the same order.

This document gives you hints, not commands. The commands, file names, and submission rules live in the Lab Document. Open both side-by-side:

- **Lab Document** → tells you *what to run*.
- **Hints Companion** (this document) → tells you *what to watch for*, *what often goes wrong*, *what the output should look like*, and *how to know when you're done*.

For every task, this document gives you:

> 💡 **Hint** — a nudge in the right direction if you get stuck or miss something subtle.

> 🔍 **What to look for** — the specific lines, fields, or details in the command output that matter.

> ⚠ **Common mistake** — a pitfall that has tripped up other learners. Read these before running the command, not after.

> ✅ **You're done when** — a self-check you can run to confirm the task is complete before moving on.

> 📋 **Sample output** — a trimmed example of what the command should produce, so you know whether you're seeing the right shape of data.

This document is for guidance only and does not need to be submitted. Submit only the deliverables listed in the Lab Document.

Task 1 — Determine Your Kali System's Network Identity

> 💡 **Hint** — `ip a s eth0` is shorthand for `ip address show eth0`. Both work. The command lists all addresses (IPv4 and IPv6) bound to the `eth0` interface.

> 💡 **Hint** — Your IP is on the line that starts with `inet ` (with a space). The line that starts with `inet6` is your IPv6 address — record IPv4 for this task.

> 🔍 **What to look for** — Three things on the `inet` line: the IPv4 address (e.g., `10.0.2.15`), the prefix length (e.g., `/24`), and the broadcast suffix (you can ignore broadcast for this task). The line just above shows the interface state — it should say `state UP`.

> ⚠ **Common mistake** — Recording the `link/ether` address (your MAC address) instead of the IP. The MAC looks like `52:54:00:12:34:56` with colons; the IP looks like `10.0.2.15` with dots.

> ⚠ **Common mistake** — Running `ip a` (without specifying `eth0`), seeing the loopback interface `lo` at the top, and recording `127.0.0.1`. That's the localhost — not a real network address.

📋 **Sample output** (your numbers will differ):

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:54:00:12:34:56 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 86342sec preferred_lft 86342sec
    inet6 fe80::5054:ff:fe12:3456/64 scope link
       valid_lft forever preferred_lft forever
```

The bold value is what you record.

✅ **You're done when** your report shows an IPv4 address (four numbers separated by dots), the interface name `eth0`, and the subnet (e.g., `/24`). You also have `task01_ip.png` saved.

Task 2 — Capture and Record Network Connectivity Data

💡 **Hint** — The `-c 4` flag means "send 4 packets, then stop." Without it, ping runs forever and you have to press `Ctrl+C` to stop it. Saving an infinite ping to a file means an infinitely growing file — avoid.

💡 **Hint** — When you run `ping ... > target_info.txt`, you will see nothing on the screen for a few seconds. That's correct. The output is going to the file, not the terminal. The terminal will look "frozen" — it's not. Just wait for the prompt to return.

💡 **Hint** — The `>` symbol means "redirect output to file, overwriting whatever was there." Use it for the first save. Use `>>` (two arrows) to append.

🔍 **What to look for in the file** — The first line should show the target IP (currently `45.33.32.156`). The last few lines should be a statistics block that says "X packets transmitted, Y received, Z% packet loss" and an average round-trip time.

⚠ **Common mistake** — Forgetting the `>` redirection, running the ping, and getting a perfect output on screen but an empty `target_info.txt`. Look at the file with `ls -l target_info.txt` — if it's `0` bytes, redirection didn't happen.

⚠ **Common mistake** — Typo in the domain name. If you see "Name or service not known" or "Temporary failure in name resolution," it's a typing or DNS issue, not a connectivity issue.

📋 **Sample output** (numbers will vary):

```
PING scanme.nmap.org (45.33.32.156) 56(84) bytes of data.
64 bytes from scanme.nmap.org (45.33.32.156): icmp_seq=1 ttl=53 time=72.1 ms
64 bytes from scanme.nmap.org (45.33.32.156): icmp_seq=2 ttl=53 time=71.4 ms
64 bytes from scanme.nmap.org (45.33.32.156): icmp_seq=3 ttl=53 time=71.8 ms
64 bytes from scanme.nmap.org (45.33.32.156): icmp_seq=4 ttl=53 time=72.0 ms

--- scanme.nmap.org ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 71.412/71.823/72.117/0.281 ms
```

The reverse DNS of `45.33.32.156` resolves to `scanme.nmap.org` itself — a hint that this is a dedicated host, not a shared CDN edge.

✅ **You're done when** `ls -l target_info.txt` shows a size greater than 100 bytes, `cat target_info.txt` shows the ping statistics, and you have `task02_ping.png` saved.

Task 3 — Explore the Target Website Through Firefox

💡 **Hint** — Treat this like an analyst, not a casual browser. As you click each link on `nmap.org`, ask: "If I were assessing this project as a third-party dependency, what does this page tell me?" Examples: the Download page shows release cadence and supported platforms; the Docs page hints at how mature and maintained the project is; the Reference Guide shows scope and capabilities.

💡 **Hint** — `scanme.nmap.org` is a single static welcome page — but its content is exactly what you need for the operator-authorization evidence. Read it slowly: it states who runs the host, what is permitted, what is forbidden, and what the rate limit is.

💡 **Hint** — Pay attention to the browser address bar across both sites. URL patterns (file extensions, path structure) tell you about the technology powering the site.

🔍 **What to look for** — Six categories of information: (1) the project / display name, (2) any contact details on `nmap.org`, (3) the kinds of pages observable (about, download, docs, reference, book, etc.), (4) any visible technology fingerprints (page generators, copyright lines, footer text), (5) the authorization notice published on `scanme.nmap.org` — including any rate limit or scope restriction, (6) any other "free" information like project history, maintainer names, or links to related infrastructure.

⚠ **Common mistake** — Downloading the Nmap installer "just to see what happens." Don't. The lab is recon only. Downloads are out of scope.

⚠ **Common mistake** — Running `nmap scanme.nmap.org` in the terminal because "the operator allows it." The operator does allow it. The lab does not — this is the reconnaissance phase that precedes scanning, not scanning itself.

⚠ **Common mistake** — Skipping the `scanme.nmap.org` page because "it's only one screen." That one screen is the authorization notice — without it your report cannot prove the activity was in scope.

📋 **Sample observations** (paraphrase, don't copy verbatim):

- Project name on `nmap.org`: "Nmap — the Network Mapper"
- Pages observable: Home, About, Download, Docs, Reference Guide, Book, Install Guide, Changelog, Bug Reports
- URLs include both static HTML (`/book/`, `/docs.html`) and the project is a long-established open-source project — useful context for third-party-risk review
- Authorization notice on `scanme.nmap.org` states: the host is set up to help people learn about Nmap, scanning is permitted via Nmap only (not exploits or DoS), and there is a rate limit of approximately a dozen scans per day per source

✅ **You're done when** your report has values for project / site name, page types observed, technology hints, and a summary of the `scanme.nmap.org` authorization notice. You have both `task03_homepage.png` and `task03_scanme_notice.png` saved.

Task 4 — Perform a Whois Query

💡 **Hint** — Run whois on the **parent domain** (`nmap.org`), not the subdomain (`scanme.nmap.org`). Whois records exist at the registered-domain level. A whois on a subdomain returns either nothing or the parent's record anyway, so save yourself the confusion.

💡 **Hint** — Whois output is long — often 100+ lines with legal disclaimers at the bottom. To find specific fields quickly:

```
grep -i "registrar:" target_whois.txt
grep -i "name server" target_whois.txt
grep -iE "creation|updated|expir" target_whois.txt
```

The `-i` flag makes the search case-insensitive.

💡 **Hint** — Field names vary by registrar. You may see "Registrar:", "Sponsoring Registrar:", or "Registrar Name:" — all mean the same thing. Same for dates: "Creation Date:" and "Created:" are equivalent.

🔍 **What to look for** — Six fields to record: (1) Registrar (the company that sold the domain), (2) Name Server(s) (the DNS provider — expect Linode for `nmap.org`), (3) Creation Date (how long the domain has existed — `nmap.org` is a long-established domain), (4) Updated Date, (5) Expiry / Expiration Date, (6) any registrant organization or country code if visible. The name servers you see here should match what you find later in Task 6.

⚠ **Common mistake** — Stopping at the first "Registrar:" line you see. The output may contain disclaimer text that *mentions* the word "registrar" without giving the value. Look for the line with a colon followed by an actual company name.

⚠ **Common mistake** — Recording `WHOIS limited match for "nmap.org"` as the registrar. That's the whois server's status header, not your answer.

📋 **Sample output** — your output will contain dozens of lines; these are the ones that matter:

```
   Domain Name: NMAP.ORG
   Registrar: <registrar name — verify on cohort refresh>
   Registrar IANA ID: ...
   Registrar Abuse Contact Email: ...
   Registry Expiry Date: ...
   Creation Date: 1999-XX-XXTXX:XX:XXZ
   Updated Date: ...
   Name Server: NS1.LINODE.COM
   Name Server: NS2.LINODE.COM
   Name Server: NS3.LINODE.COM
   Name Server: NS4.LINODE.COM
   Name Server: NS5.LINODE.COM
```

The exact registrar and current dates change over time; instructors should refresh this section per cohort. The name-server set should be stable.

✅ **You're done when** `target_whois.txt` exists with size > 200 bytes, your report has values for registrar, name servers, and at least one date field, and `task04_whois.png` is saved.

Task 5 — Conduct DNS Reconnaissance with Nslookup

💡 **Hint** — Once you type `nslookup` by itself (no arguments), you enter "interactive mode." The prompt changes to `>`. From there, just type a domain — no command prefix needed.

💡 **Hint** — Type `exit` at the `>` prompt to leave interactive mode. `Ctrl+C` also works but can leave the terminal in a weird state — prefer `exit`.

💡 **Hint** — The first two lines of the response show the resolver (your local DNS server). The next block shows the actual answer.

🔍 **What to look for** — (1) The `Server:` line at the top — this is the DNS resolver your Kali VM is configured to use (often your gateway, e.g., `10.0.2.3` in VirtualBox). (2) The `Address:` line — your resolver's IP. (3) A "Non-authoritative answer:" label below — this just means your resolver got the answer from its cache or a recursive query, not directly from the authoritative server. (4) The `Name:` and `Address:` lines under that — the target's hostname and IPv4 address.

⚠ **Common mistake** — Typing `nslookup scanme.nmap.org` on a single line, which runs in non-interactive mode and exits immediately. The lab specifically asks for interactive mode — type `nslookup` alone, then the domain at the prompt.

⚠ **Common mistake** — Confusing the resolver's address (the `Server:` line) with the target's address. The resolver is your DNS service; the target is what you're looking up.

📋 **Sample session**:

```
$ nslookup
> scanme.nmap.org
Server:     10.0.2.3
Address:    10.0.2.3#53

Non-authoritative answer:
Name:   scanme.nmap.org
Address: 45.33.32.156

> exit
$
```

The "Non-authoritative" label means the answer came from a recursive resolver, not directly from the domain's own name server. That's normal for a default lookup.

✅ **You're done when** your screenshot shows the `Server:` line, the `Non-authoritative answer:` block, and an A record for `scanme.nmap.org`. Your report has the resolver IP and the target A record.

Task 6 — Query Authoritative DNS Records

💡 **Hint** — The `server` command inside `nslookup` changes which DNS server you query. You're switching from your local resolver to one of the domain's authoritative name servers. After this, the answers will not be labeled "Non-authoritative" — that tells you the switch worked.

💡 **Hint** — `set type=X` changes the kind of record you're asking for. After the change, just type the domain — don't repeat `set`.

💡 **Hint** — Query CNAME against `www.nmap.org`, not `nmap.org` itself. Root domains rarely have CNAME records (it's a DNS rule). For `www.nmap.org`, if there's no CNAME, you'll see no answer or "** server can't find ... NXDOMAIN" — that's a valid finding. Record it as "no CNAME returned."

💡 **Hint** — MX records have two parts: a priority number (lower = preferred) and a hostname. `1 aspmx.l.google.com` means priority 1 (highest), mail server is Google. For `nmap.org` you'll see Google Workspace mail servers at priorities 1, 5, 5, 10, 10 — a typical Google Workspace MX layout.

🔍 **What to look for** — Five sets of values: (A) IPv4 address for `nmap.org`, (MX) five mail exchangers all pointing to Google with priorities 1/5/5/10/10, (NS) five Linode name servers (`ns1.linode.com` through `ns5.linode.com`), (CNAME) usually empty for this target, (SOA) "Start of Authority" — primary NS is `ns1.linode.com`, responsible mailbox is `hostmaster.nmap.com` (note: `.com`, not `.org` — that sister domain is itself a finding worth noting).

⚠ **Common mistake** — Typing `set type=MX nmap.org` on one line. That doesn't work in nslookup. The `set` command and the query are two separate inputs.

⚠ **Common mistake** — Reading the SOA "responsible mailbox" as a literal email. It's a DNS-encoded address: `hostmaster.nmap.com.` means `hostmaster@nmap.com` in normal notation.

⚠ **Common mistake** — Querying NS or SOA without first switching `set type`. Without that, you'll keep getting A records.

📋 **Sample session** (trimmed for space — your screenshot should show all five record types):

```
$ nslookup
> server ns1.linode.com
Default server: ns1.linode.com
Address: ...#53

> set type=MX
> nmap.org
Server:     ns1.linode.com
...
nmap.org    mail exchanger = 1 aspmx.l.google.com.
nmap.org    mail exchanger = 5 alt1.aspmx.l.google.com.
nmap.org    mail exchanger = 5 alt2.aspmx.l.google.com.
nmap.org    mail exchanger = 10 aspmx2.googlemail.com.
nmap.org    mail exchanger = 10 aspmx3.googlemail.com.

> set type=SOA
> nmap.org
Server:     ns1.linode.com
...
nmap.org
    origin = ns1.linode.com
    mail addr = hostmaster.nmap.com
    serial = 2021000018
    refresh = 14400
    retry = 14400
    expire = 1209600
    minimum = 3600
```

Note the MX records point to Google Workspace and the SOA mailbox is `hostmaster.nmap.com` (not `.org`) — both are findings worth noting in your report.

✅ **You're done when** your report shows values for all five record types (or "none returned" for CNAME), and your screenshot covers the entire authoritative session from `server` to `exit`.

Task 7 — Capture DNS Information Using Dig

💡 **Hint** — The single arrow `>` means "overwrite." The double arrow `>>` means "append." Use `>` on the **first** dig only, then `>>` on every dig after that. If you use `>` twice, the second one deletes the first one's result.

💡 **Hint** — Want to add helpful comments between sections? Add separator lines:

```
echo "=== MX RECORDS ===" >> target_dns.txt
dig nmap.org MX >> target_dns.txt
```

This makes `cat` -ing the file much easier to read later.

💡 **Hint** — Each dig produces a multi-section response with a `;; QUESTION SECTION:`, a `;; ANSWER SECTION:`, and an `;; AUTHORITY SECTION:`. The lines starting with `;;` are comments — they're decorative, not actual DNS data. The data lines (no leading `;`) below the section headers are the records.

🔍 **What to look for in the file** — Five separate `;; ANSWER SECTION:` blocks (one per query), each followed by one or more record lines. The data should match what you saw in Task 6.

⚠ **Common mistake** — Using `>` on every command. Result: `target_dns.txt` contains only the last command's output (SOA only). Fix: re-run starting with `dig nmap.org > target_dns.txt` and use `>>` for the rest.

⚠ **Common mistake** — Verifying with `cat target_dns.txt` only at the very end and finding it's nearly empty. Run `ls -l target_dns.txt` after each `>>` — the size should grow each time.

📋 **Sample output** — one of the five sections in `target_dns.txt`:

```
; <<>> DiG 9.18.x <<>> nmap.org MX
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 12345
;; flags: qr rd ra; QUERY: 1, ANSWER: 5, AUTHORITY: 0, ADDITIONAL: 1

;; QUESTION SECTION:
;nmap.org.          IN  MX

;; ANSWER SECTION:
nmap.org.       3600    IN  MX  1 aspmx.l.google.com.
nmap.org.       3600    IN  MX  5 alt1.aspmx.l.google.com.
nmap.org.       3600    IN  MX  5 alt2.aspmx.l.google.com.
nmap.org.       3600    IN  MX  10 aspmx2.googlemail.com.
nmap.org.       3600    IN  MX  10 aspmx3.googlemail.com.

;; Query time: 76 msec
;; SERVER: 10.0.2.3#53(10.0.2.3)
;; WHEN: ...
;; MSG SIZE  rcvd: 161
```

The TTL (`3600`) is how long resolvers cache this record, in seconds — 1 hour here.

✅ **You're done when** `target_dns.txt` exists with size > 500 bytes, `grep -c "ANSWER SECTION" target_dns.txt` returns at least 4 (CNAME may produce an empty answer, which is fine), and your screenshot is saved.

Task 8 — Practice Safe Search Engine Reconnaissance

💡 **Hint** — Search operators compose. `site:example.com filetype:pdf intitle:report` combines three filters. The more you stack, the more targeted the result.

💡 **Hint** — Quotation marks force an exact phrase. `"contact our team"` looks for those exact words in that exact order. Without quotes, the engine may rearrange or drop words.

💡 **Hint** — The minus operator removes terms. `example -wikipedia` excludes any result that mentions Wikipedia. Useful for filtering noise.

💡 **Hint** — Even though our recon target is `nmap.org`, we don't run search operators against it. Two reasons: (1) the lab structure separates "operator practice" from "target recon," and (2) the target is heavily indexed because of its long-established educational presence, which makes the results noisy. Stick to the approved demo domains for this task.

🔍 **What to look for** — For each query, note: (1) the operator used, (2) the kind of results returned (web page, PDF, image, etc.), (3) what useful public information appeared, (4) the security value — what could a defender or attacker learn from this technique applied to a real organization?

⚠ **Common mistake** — Running searches for passwords, credentials, leaked databases, or any real person's name. Out of scope. Will fail the rubric line.

⚠ **Common mistake** — Documenting only the queries without saying what they revealed. The report grades both the operator use and the analysis.

✓ **You're done when** your report lists at least four distinct queries, each with the operator(s) used, the result type, and a one-line interpretation of its security value. Two screenshots are saved.

Task 9 — Use an Approved Website OSINT Lookup Tool

💡 **Hint** — Enter the parent domain (`nmap.org`), not the subdomain. These tools typically index registered domains.

💡 **Hint** — Different OSINT tools surface different fields. Not all of them will be available — record whichever the tool actually shows. Three populated fields is a good target; more is fine.

💡 **Hint** — Pay attention to the "first seen" date or domain age. A domain that's existed for 25+ years (as `nmap.org` has) and a brand new domain have very different risk profiles for a third-party-risk review.

🔍 **What to look for** — Five common OSINT fields: hosting provider (expect Linode for this target), server technology (e.g., Nginx, Apache), CMS or framework (the Nmap site is largely static HTML — a finding in itself), first-seen or domain-age date, JavaScript libraries or analytics tools used.

⚠ **Common mistake** — Treating the OSINT tool's output as ground truth without questioning it. These tools rely on past crawls — their data can be outdated. Cross-check anything important against your dig results.

✓ **You're done when** your report has at least three OSINT fields with values, and `task09_osint.png` is saved.

Task 10 — Perform Public Data Awareness Review

💡 **Hint** — Think like a phisher. Which combinations of data fields would help craft a convincing impersonation email? Name + email + manager's name is a classic example. Name + role + active project name is another.

💡 **Hint** — Think like a defender. For each risk you identify, what's one practical control that reduces it? Examples: role-based shared inboxes instead of named personal addresses, social media training, removing org-chart pages from public sites, separating internal-only project codenames from external comms.

💡 **Hint** — Don't conflate "available" with "exposed." A name on a corporate site isn't a vulnerability by itself. The risk is in *combinations*: name + email + role + manager's name + active project = a phishing toolkit.

🔍 **What to look for** — Four or more *categories* of information (not individual fields): personal identifiers, employment context, project context, infrastructure context, online presence. For each, articulate the misuse risk and a reduction strategy.

⚠ **Common mistake** — Looking up real people who share names with the fictional dataset. The data is fictional — leave it that way.

⚠ **Common mistake** — Listing categories without explaining the risk. "Email addresses" by itself is not analysis. "Email addresses + visible org chart enable targeted phishing of new hires by impersonating their direct manager" is analysis.

✓ **You're done when** your report lists at least four data categories, each paired with at least one misuse risk and at least one reduction strategy.

Task 11 — Organize Reconnaissance Evidence

💡 **Hint** — If you have `tree` installed, `tree ~/recon_lab/` produces a nicer visualization than `ls -R`. Either is acceptable.

💡 **Hint** — Filenames matter. The grader looks for specific names. If your screenshots default to `Screenshot from 2026-05-11 14-22-31.png`, rename them: `mv "Screenshot from 2026-05-11 14-22-31.png" task01_ip.png`.

⚠ **Common mistake** — Leaving evidence in `~/Downloads` or `~/Desktop` because that's where screenshots default to. Move them: `mv ~/Pictures/Screenshot* ~/recon_lab/screenshots/` (path varies by tool).

⚠ **Common mistake** — Saving the report file outside `~/recon_lab/`. The grader expects every deliverable inside that folder. Move it: `mv ~/Documents/recon_report.docx ~/recon_lab/`.

✅ **You're done when** `ls -R ~/recon_lab/` shows all three text files in `~/recon_lab/`, the report file, and every required screenshot inside `~/recon_lab/screenshots/`.

Task 12 — Document and Correlate Reconnaissance Findings

💡 **Hint** — Connect each finding to a "so what." Listing "Hosting provider: Linode" tells the reader nothing. "Hosting provider: Linode (Akamai) — third-party-risk review should reference Akamai/Linode's published compliance posture when assessing infrastructure controls" is analysis.

💡 **Hint** — Keep the report tight. Two to three sentences per finding is plenty. Long is not better — clear is better.

💡 **Hint** — For *vulnerability management* implications, ask: "Now that we know X, what should the vuln-management team scan, monitor, or include in scope?" Example: knowing the MX is Google Workspace means email-borne phishing controls and DMARC/SPF/DKIM config become relevant when validating mail from this project.

💡 **Hint** — For *third-party risk* implications, ask: "Now that we know X, which third parties does the organization depend on, and what's their security posture?" Examples: hosting (Linode), DNS (Linode), email (Google), domain registrar (whichever your whois showed).

💡 **Hint** — The "Recommended next steps" section is short but important. Three concrete suggestions are enough. Examples: "Verify the Nmap Project's published release-signing process before approving the Nmap binary for internal distribution," "Add Linode and Google Workspace to the third-party register and confirm their compliance attestations are current," "If Nmap is used inside internal scans, document the segmentation/authorization controls that govern its use."

⚠ **Common mistake** — Leaving template placeholders (like `_Your name_`) unedited. Read the report top-to-bottom one final time before submitting.

⚠ **Common mistake** — Writing the report from memory after closing all the output files. Keep the .txt files and screenshots open while you write — accuracy matters more than speed.

⚠ **Common mistake** — A "Key risks" section that just repeats facts ("the project uses Google Workspace"). Risk is a fact *plus* a consequence ("the project uses Google Workspace; SPF/DKIM/DMARC posture should be confirmed before treating any email apparently from the project as authentic").

✅ **You're done when** every numbered section of the template has content, no placeholder underscores remain, and the file is saved as `recon_report.docx` or `recon_report.pdf` in `~/recon_lab/`.

DNS Record Type Cheat Sheet

Type	What it tells you
A	The IPv4 address for a hostname. The headline answer for "where does this hostname live?"
AAAA	The IPv6 address — same idea as A, just for IPv6.
MX	Mail exchanger — which mail servers handle email for the domain, and in what priority order. Reveals the email provider (Google Workspace, Microsoft 365, on-premise Exchange, etc.).
NS	Name server — which DNS servers are authoritative for the domain. Reveals the DNS provider.
CNAME	Canonical name — an alias from one name to another. Often used for CDN endpoints or subdomain pointers.
SOA	Start of Authority — the "master record" for the zone. Contains the primary NS, the responsible mailbox, serial number, and timing values for cache refresh.
TXT	Free-form text — used for SPF, DKIM, DMARC, domain ownership verification, and many other purposes. Not queried in this lab but worth knowing.
PTR	Reverse pointer — maps an IP back to a hostname. Used in reverse-DNS lookups.

Command Quick Reference

Command	Purpose
`ip a s eth0`	Show IP info for the eth0 interface
`ping -c 4 <host>`	Send 4 ICMP echoes to host
`ls -l <file>`	Long-format directory listing (size, date, permissions)
`cat <file>`	Display the entire file content
`tail <file>`	Display the last 10 lines (useful for long output files)
`grep -i <text> <file>`	Case-insensitive search for text in a file
`whois <domain>`	Query domain registration data
`nslookup`	Enter interactive DNS query mode
`nslookup <host>`	One-shot DNS lookup (non-interactive)
`dig <domain>`	Query A record for the domain
`dig <domain> MX`	Query a specific record type
`> file`	Redirect output to file (overwrite)
`>> file`	Redirect output to file (append)

Troubleshooting

"ping: name or service not known" DNS isn't resolving. Try `host scanme.nmap.org` to confirm DNS works at all, and `cat /etc/resolv.conf` to see your configured resolver. If `host` fails too, ask your instructor.

"whois: command not found" Whois isn't installed: `sudo apt-get install -y whois`.

Nslookup gets stuck after `server …` You may have typed an unreachable name server. Press `Ctrl+D` to exit, then start over with `ns1.linode.com`, `ns2.linode.com`, or any of `ns3-5.linode.com`.

`target_dns.txt` is much smaller than expected You probably used `>` (overwrite) instead of `>>` (append) on later commands. Re-run Task 7 from the start.

Firefox can't load `nmap.org` Check basic connectivity: `ping -c 2 scanme.nmap.org`. If ping works but the browser doesn't, check Firefox's proxy settings (Preferences → Network → Settings → No proxy).

Screenshot tool doesn't save where you expect After taking a screenshot, find it with:

```
find ~ -name "Screenshot*" -mmin -5
```

Then move it: `mv "<found path>" ~/recon_lab/screenshots/taskNN_xxx.png`.

Lost VM keyboard focus Click inside the VM window to give it focus. To release back to Windows 11, press `Ctrl + Alt`.

Cloud portal session timed out Re-login. Your Kali VM keeps running server-side — pick up where you left off.

Final Pre-Submission Checklist

Before zipping and submitting, confirm:

- [ ] `target_info.txt` exists and contains ping statistics.
- [ ] `target_whois.txt` exists and contains registrar + name servers.
- [ ] `target_dns.txt` exists with five separate dig responses.
- [ ] `recon_report.docx` (or `.pdf`) has every section filled in.
- [ ] `screenshots/` contains all 12 expected PNGs, each named correctly.
- [ ] No screenshot shows an Nmap or scanner output, a download attempt, a form submission, or any active probing of the target.
- [ ] No report section contains placeholder underscores or "TODO".
- [ ] Findings in the report are interpreted, not just listed.

When all boxes are checked, you're ready to submit. Good work.

End of Hints Companion.